Implementación del Núcleo 5G y de la Red de Acceso sin Radiofrecuencia (Zero-RF): Despliegue Contenerizado de Open5GS y srsRAN con Análisis de Protocolos

Jose David Rodríguez Pinilla*
*Universidad Icesi, Cali, Colombia
Código: A00407151 Correo: jdrpjd@gmail.com
Curso: Sistemas y Comunicaciones 2 — Docente: Andrés Navarro Cadavid, PhD

***Resumen*—Este informe documenta la Fase 2 (Implementación del Core y de la Red de Acceso, Zero-RF) del proyecto de curso que despliega una red 5G Standalone (SA) con *srsRAN Project* como red de acceso y *Open5GS* como núcleo 5G con servicios virtualizados tipo NFV. El despliegue se automatizó con Docker Compose en cinco contenedores, con una configuración única versionable y un banco de pruebas en Python que guarda resultados trazables. El núcleo ejecuta once funciones de red con la base de datos MongoDB aprovisionada con el perfil 5G del suscriptor, y el gNB se enlaza al AMF por NGAP/SCTP con el túnel GTP-U estable. Se habilitó el cifrado NAS 128-NEA2 con integridad 128-NIA2 tras identificar y corregir, actualizando el srsUE, un defecto de descifrado que antes obligaba a usar NEA0, y se parametrizó la seguridad AS del gNB. El análisis de las trazas del registro y de la sesión PDU, automatizado con *tshark*, extrae el SUCI, el vector 5G-AKA, los algoritmos negociados, los TEID de N3 contrastados entre NGAP y PFCP, y la duración de cada procedimiento: el registro completo hasta la sesión PDU tardó 471.6 ± 32.3 ms (IC 95 %, $n = 5$). Con cinco corridas por prueba se midió un RTT al UPF de 31.9 ± 0.9 ms, un caudal TCP de 48.6 ± 1.4 Mbps en descendente y 11.6 ± 0.5 Mbps en ascendente, un máximo UDP descendente de 56.3 ± 1.2 Mbps y treinta minutos de operación sin cortes. El barrido de MCS fijo muestra que el caudal es el 54–62 % de la cota de TS 38.306 en todos los MCS, lo que atribuye la brecha a la sobrecarga de capa física y de planificación y no a la CPU. Se documentan quince fallas corregidas, las decisiones técnicas y la preparación de la Fase 3.**

***Palabras clave*—5G NR, Standalone, Open5GS, srsRAN, ZeroMQ, Docker, NGAP, NAS-5GS, PFCP, GTP-U, 5G-AKA, NEA2, análisis de protocolos.**

I. Introducción

I-A. Contexto

En la Fase 1 se seleccionaron Open5GS y srsRAN Project, se definió la arquitectura lógica con las interfaces N1, N2, N3, N4 y N6 y se validó el banco de simulación sin radiofrecuencia basado en ZeroMQ (ZMQ) [12], [14], [15]. Esa validación se hizo con una instalación manual sobre WSL2 y dejó pendientes la conectividad a Internet del UE, las pruebas P7–P9 y el análisis de las trazas. La Fase 2 convierte ese banco en un despliegue reproducible y lo valida a fondo, todavía sin hardware de radio.

I-B. Objetivos

El **objetivo general de la Fase 2** es desplegar de forma lógica el sistema completo (núcleo, gNB y UE) sin hardware de radio y verificar su señalización NAS/AS y su plano de usuario. Los objetivos específicos son:

1. Desplegar el núcleo Open5GS con todas sus funciones de red y la base de datos de suscriptores configurada con seguridad 5G.
2. Configurar el gNB de srsRAN y vincularlo al núcleo por NGAP/SCTP, con un túnel GTP-U estable hacia el UPF.
3. Capturar e interpretar campo a campo la señalización del registro y de la sesión PDU.
4. Medir latencia, caudal, MCS, estabilidad y consumo de recursos con rigor estadístico, y contrastarlos con la teoría.
5. Documentar las decisiones técnicas, las fallas y su solución, y preparar la integración con SDR (Fase 3).

I-C. Alcance y trazabilidad con la rúbrica

La radio sigue siendo virtual (ZMQ) y el UE es el emulador srsUE; el SDR y el terminal comercial se integran en la Fase 3. La Tabla I relaciona cada criterio de la rúbrica de la Fase 2 con la sección que lo atiende.

II. Arquitectura del despliegue

II-A. Contenerización y red

Cada componente corre en su propio contenedor Docker, orquestado con Docker Compose [16] (Fig. 1). Docker Compose actúa como un orquestador NFV mínimo: describe las funciones como unidades desplegables, las conecta y las reinicia si fallan, aunque sin escalado ni autocuración avanzada (que en producción aportarían Kubernetes, OSM u ONAP). Los contenedores comparten la red `red5g_net` (10.53.0.0/24). Esta subred se eligió fuera del rango 172.17–172.31 que Docker reparte a otros proyectos, tras un choque real de direcciones (falla F2-12).

Tabla I
TRAZABILIDAD ENTRE LA RÚBRICA DE LA FASE 2 Y ESTE INFORME

Criterio	Evidencia incluida	Secc.
Core Open5GS (ABET 2)	11 NF activas y registradas en el NRF, MongoDB aprovisionado con el perfil 5G, seguridad NAS NIA2/NEA2 y AS, estabilidad de 30 min	III
gNB srsRAN (ABET 1, 2)	NG Setup por NGAP/SCTP, parámetros de celda, túnel GTP-U con TEID verificados, caudal y MCS medidos frente a la teoría	IV, VI
Análisis de protocolos (ABET 1)	Registration y PDU Session campo a campo (SUCI, RAND/AUTN/RES*, ABBA, NEA/NIA, TEID, PDR/FAR/QER, IP), diagrama de secuencia medido y tiempos con IC 95 %	V
Gestión técnica (ABET 4)	Decisiones justificadas, 15 fallas resueltas, flujo de trabajo (git, bitácora, CI), optimización de recursos y preparación de la Fase 3	VII

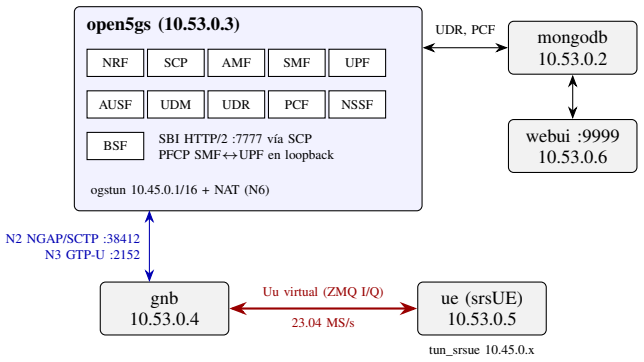


Figura 1. Despliegue contenerizado de la Fase 2. El servicio `gnb-sdr` (perfil `sdr`, 10.53.0.7) queda preparado para la Fase 3 con la misma imagen del gNB.

II-B. Configuración única y reproducibilidad

Todos los valores del despliegue (PLMN, TAC, *slice*, DNN, algoritmos de seguridad, suscriptor, direcciones, rango de MCS, memoria del gNB) se definen una sola vez en un archivo `.env`. Los archivos de configuración de cada contenedor son plantillas con marcadores `${VARIABLE}` que se rellenan al arrancar, y el arranque aborta si queda un marcador sin valor. El archivo `.env` no se versiona porque en la Fase 3 contendrá las claves de la SIM física (Ley 1581 de 2012 [18]); en el repositorio se versiona la plantilla `.env.example`. La Tabla II fija las versiones, que se compilan desde el código fuente para que el despliegue no cambie sin aviso.

III. NÚCLEO OPEN5GS

III-A. Funciones de red y arranque

El contenedor del núcleo arranca las once funciones de red (NRF, SCP, AMF, SMF, UPF, AUSF, UDM, UDR, PCF, NSSF y BSF). Cada una escucha su interfaz SBI en una dirección de *loopback* propia (por ejemplo, NRF en 127.0.0.10 y AMF en 127.0.0.5), y todas se comunican por HTTP/2 a través del SCP (127.0.0.200:7777). Hacia la red Docker solo se exponen NGAP del AMF (10.53.0.3:38412/SCTP) y GTP-U del

Tabla II
COMPONENTES Y VERSIONES FIJADAS

Componente	Versión	Observación
Núcleo 5G	Open5GS v2.8.0 (fuente)	11 NF en un contenedor
Base de datos	MongoDB 6.0	Volumen persistente
gNB	srsRAN Project `4bf1543936` (25.10)	ZMQ y UHD en un binario
UE	srsUE 26.10 (`06f0b1e23`)	USIM por software
Orquestación	Docker Compose (Docker 29.9)	WSL2, 8 CPU, 8 GB
Análisis	Python 3.12, tshark 4.6.9	Banco sin dependencias

Tabla III
PERFIL DEL SUSCRIPTOR APROVISIONADO EN MONGODB

Campo	Valor
SUPI (IMSI)	001010123456780 (PLMN de pruebas 001/01)
Autenticación	5G-AKA con Milenage; K y OPc de prueba; AMF = 8000; SQN inicial 0
Slice (S-NSSAI)	SST = 1 (eMBB), sin SD; *slice* por defecto
DNN y tipo de PDU	`internet`, IPv4
QoS por defecto	5QI = 9, ARP 8 (con capacidad y vulnerabilidad de preferencia)
Session-AMBR / UE-AMBR	1 Gbps DL / 1 Gbps UL

UPF (10.53.0.3:2152/UDP). El *entrypoint* configura la interfaz `ogstun` (10.45.0.1/16), el reenvío IP y el NAT de N6. Si una función de red muere, el contenedor termina para que Docker lo reinicie completo, y el *healthcheck* declara el núcleo sano solo cuando el AMF escucha NGAP. El gNB arranca únicamente después (`depends_on: service_healthy`); en la versión inicial se sondeaba NGAP por TCP, lo que fallaba porque NGAP usa SCTP (F2-5) [11].

III-B. Base de datos y perfil del suscriptor

MongoDB se inicializa con un *script* que crea el suscriptor de prueba a partir de `.env` (Tabla III). El esquema sigue el de Open5GS 2.8: la QoS de la sesión va en el campo `qos`. El esquema antiguo (`pcn`) producía un registro exitoso con la sesión PDU rechazada por "No 5QI" (F2-8). El *script* borra e inserta por IMSI, así que repetir el alta es seguro. El mismo archivo se reutiliza para dar de alta la SIM física de la Fase 3 (`python -m banco suscriptor`), de modo que el esquema existe en un solo lugar. Las claves pasan al contenedor por variables de entorno y nunca aparecen en la línea de comandos ni en los resultados.

III-C. Seguridad 5G

La seguridad del estrato de no acceso (NAS) entre el UE y el AMF se configura por orden de preferencia de algoritmos [7]. La integridad usa 128-NIA2 (AES-CMAC) y el cifrado 128-NEA2 (AES-CTR). En las primeras pruebas el cifrado debía quedarse en NEA0 (sin cifrar): con NEA1, NEA2 o NEA3 el UE se liberaba justo después del *Security Mode Command*. La

Tabla IV
PARÁMETROS DE LA CELDA NR Y DEL ENLACE CON EL NÚCLEO

Parámetro	**Valor**
Banda / modo dúplex	n3 / FDD
DL-ARFCN / SSB-ARFCN	368500 (1842.5 MHz) / 368410
Ancho de banda / SCS	20 MHz / 15 kHz ($\mu = 0$), 106 PRB
PCI / TAC / PLMN	1 / 7 / 001-01
Tabla MCS (PDSCH, PUSCH)	`qam64`, MCS 0–28
PRACH	`prach_config_index` = 1
Muestreo ZMQ	23.04 MS/s
N2 (NGAP)	SCTP 10.53.0.4 $\rightarrow$ 10.53.0.3:38412
N3 (GTP-U)	10.53.0.4:2152 $\leftrightarrow$ 10.53.0.3:2152
Buffer pool	131 072 segmentos de 2048 B (256 MiB)

causa estaba en el srsUE 25.10: para descifrar los mensajes NAS de 5G recalculaba el contador NAS COUNT con la posición del número de secuencia que usa LTE, pero en 5G ese byte es el último de la MAC. El *keystream* resultante era erróneo y el *Registration Accept* no se podía descifrar (issue #1392 de srsRAN_4G, falla F2-15). La versión 26.10 del srsUE [13] incluye la corrección y, con ella, el registro completa con NEA2 + NIA2, como verifica la traza de la Sección V.

La seguridad del estrato de acceso (AS) protege RRC y PDCP en la interfaz radio y es independiente de la NAS. srsRAN Project prefiere por defecto `nea0`, es decir, radio sin cifrar aunque el NAS sí lo esté. En este despliegue se parametrizó con `GNB_NEA_PREF_LIST=nea2,nea1,nea3,nea0`. Con esa preferencia el UE registra y pasa tráfico. La comparación de los tiempos de registro con `nea0` y `nea2` no muestra una penalización medible (Sección V). El gNB solo registra el algoritmo AS negociado en el nivel de depuración, que puede volcar claves derivadas; esa evidencia queda pendiente para una prueba controlada.

IV. GNB SRSRAN Y ENLACE CON EL NÚCLEO

IV-A. Configuración de la celda

La Tabla IV resume la celda. La radio virtual ZMQ intercambia muestras I/Q entre el gNB y el UE por TCP: cada extremo enlaza su puerto de transmisión y se conecta al del otro. En la primera versión los puertos estaban cruzados y no había radio (F2-6). El rango de MCS del planificador se toma de `.env`; al fijar el mínimo igual al máximo se obtiene la prueba P8.

IV-B. NG Setup y túnel GTP-U

Al arrancar, el gNB abre la asociación SCTP con el AMF y envía *NGSetupRequest* [4] con su identificador, el TAC y el PLMN soportados. El AMF responde *NGSetupResponse* con su nombre, la GUAMI y los S-NSSAI soportados. En las cinco corridas de la prueba de registro el procedimiento tardó 18.7 ± 7.0 ms. Un error de PLMN entre el AMF y el gNB produce *NGSetupFailure* con la causa `unknown-PLMN-or-SNPN` (F2-3).

El túnel GTP-U de N3 requería que el UPF y el gNB enlazaran las direcciones de la red Docker. Con los valores por defecto (0.0.0.0 en el UPF y 127.0.0.1 en el gNB) el plano de usuario era inalcanzable (F2-4). Con la corrección, los TEID que intercambian NGAP y PFCP coinciden (Sección V), y durante 30 minutos de tráfico continuo no hubo un solo corte (Sección VI).

IV-C. Ruta del UE hacia Internet

El pendiente de la Fase 1 era la conectividad del UE a Internet. Dentro del contenedor, la ruta por defecto de Docker ganaba a la del túnel 5G, de modo que el ping a Internet "funcionaba" sin pasar por la red 5G (F2-7). El contenedor del UE reemplaza la ruta por defecto por `tun_srsue` en cuanto recibe dirección y la vigila de forma permanente. Así, todo el tráfico de usuario recorre Uu, N3 y N6.

IV-D. Memoria del gNB

El gNB ocupaba 2.44 GB con un solo UE. En el código del *commit* fijado, el *buffer pool* reserva por defecto $1\,048\,576 \times 2048$ B = 2 GiB al arrancar. Una cola RLC llena de 16 384 SDU de 1500 B cabe en unos 32 MiB, así que se redujo a 256 MiB. El gNB pasó a 0.69 GB (72 % menos) y el caudal TCP descendente no cambió (49.7 ± 1.2 frente a 48.6 ± 1.4 Mbps). El ahorro resultó decisivo: con la memoria de Windows al límite las mediciones perdían precisión (Sección VI).

V. ANÁLISIS DE PROTOCOLOS

V-A. Método de captura

La captura se hace en el contenedor del núcleo con `tcpdump -i any`, que ve N2 y N3 en la interfaz de la red Docker y N4 y la SBI en *loopback*. El filtro es `sctp or udp port 2152 or udp port 8805 or tcp port 7777`. El banco recrea juntos el gNB y el UE para capturar el registro desde el principio. Recrear los contenedores es necesario: un `docker compose restart` dejaba el enlace ZMQ sin muestras (F2-13). La traza se interpreta con *tshark* [17] mediante un analizador del banco (`python -m banco protocolos`) que extrae los campos de cada mensaje, contrasta los TEID entre NGAP y PFCP y mide la duración de cada procedimiento. La Fig. 2 muestra la secuencia medida en una de las trazas.

V-B. Registro: identidad, autenticación y seguridad

La Tabla V resume los campos extraídos de la traza.

Identidad. El *Registration Request* lleva un SUCI con formato IMSI, MCC 001, MNC 01, indicador de enrutamiento 0000, esquema de protección nulo (0) y MSIN 0123456780. El esquema nulo deja el MSIN legible en la traza, lo que es aceptable en un laboratorio con PLMN de pruebas pero no en una red comercial, donde se usaría el perfil A o B de ECIES con la clave pública de la red local [7]. El UE anuncia soporte de 5G-EA0 a 5G-EA3 y de 128-5G-IA1 a IA3 (sin IA0).

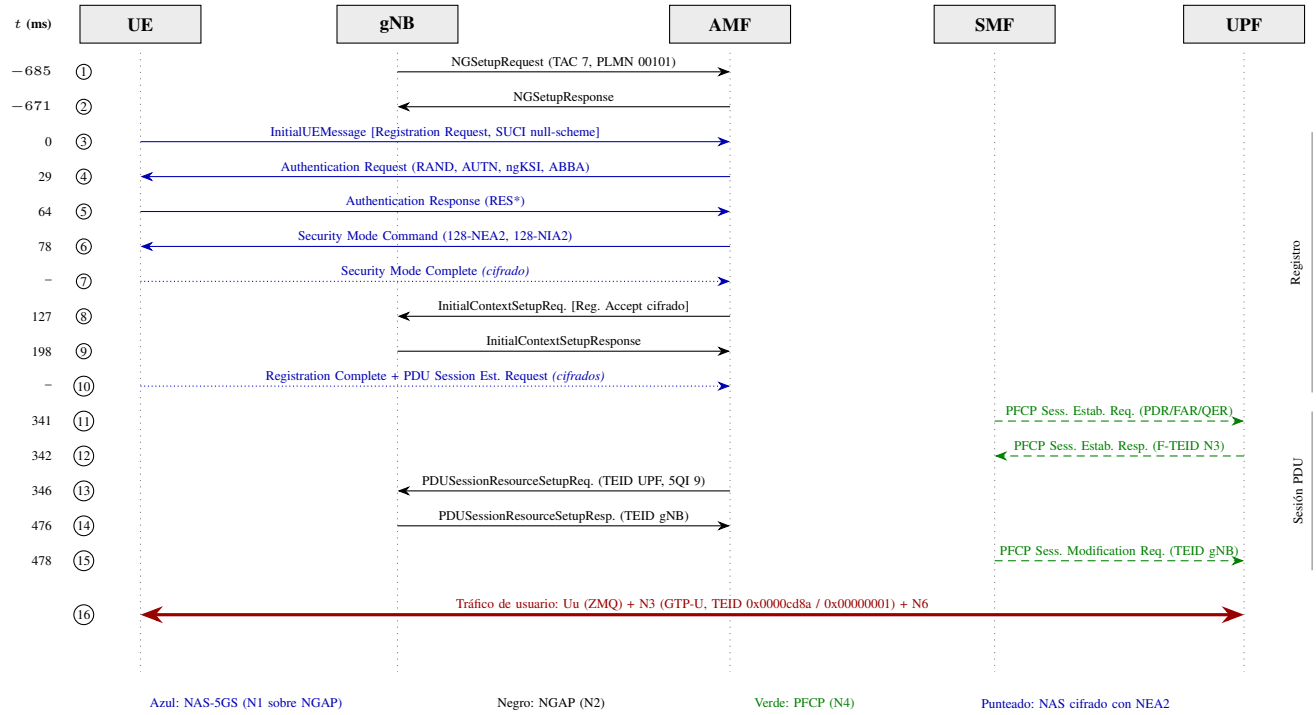


Figura 2. Secuencia medida del registro y de la sesión PDU (traza del 2 de noviembre de 2026, corrida 1). Tiempos relativos al *Registration Request*. La señalización SBI (HTTP/2) entre el AMF, la AUSF/UDM y el SMF también está en la traza, pero se omite por claridad.

5G-AKA. El *Authentication Request* transporta RAND (128 bits) y AUTN, este último desglosado en SQN⊕AK (48 bits), el campo AMF = 8000 y MAC (64 bits). El bit de separación del campo AMF está activo, como exige 5G-AKA para distinguir los vectores 5G de los de EPS. ABBA vale 0000. El UE calcula RES* a partir de K, OPc y RAND (Milenage) [10], y lo devuelve en el *Authentication Response*. La AUSF lo compara con el XRES* que generó el UDM. Este intercambio tardó 29.9 ± 5.0 ms.

Seguridad NAS. El *Security Mode Command* selecciona 128-NEA2 y 128-NIA2 y repite las capacidades de seguridad del UE para evitar ataques de degradación. A partir de ahí los mensajes NAS llevan cabecera de seguridad tipo 2 (integridad y cifrado) [3]: en cada traza hay 7 mensajes cifrados, entre ellos el *Registration Accept*, el *Registration Complete* y el intercambio de la sesión PDU. Su contenido no puede leerse en la captura del núcleo. Para inspeccionarlo hay que usar la captura NAS que guarda el propio UE o, temporalmente, NEA0. En las trazas de octubre, con NEA0, la cabecera era también tipo 2 pero el contenido era legible. El analizador distingue ambos casos según el algoritmo negociado.

V-C. Sesión PDU y plano de usuario

La petición de sesión PDU llega cifrada dentro de un *UplinkNASTransport* [2]. El AMF la entrega al SMF por la SBI, y el SMF instala las reglas en el UPF por N4 con una *PFCP Session Establishment Request*. En términos de redes definidas por software (SDN), esa petición es la evidencia de la separación de planos (CUPS) [5]. El SMF actúa como controlador e instala reglas *match-action* en el plano de datos:

- **PDR 1** (origen *Core*, interfaz N6, *network instance* `internet`): tráfico de bajada dirigido a la IP del UE (10.45.0.5), asociado a la FAR 1, la QER 1 y la URR 1.
- **PDR 2** (origen *Access*, N3, con *CHOOSE ID* para que el UPF asigne el F-TEID): tráfico de subida con QFI 1. Retira la cabecera GTP-U/UDP/IP y lo reenvía con la FAR 2.
- **PDR 3 y 4**: reglas de control entre el UPF y el plano de

Tabla V
CAMPOS EXTRAÍDOS DE LA TRAZA DEL REGISTRO (CORRIDA 1)

Campo	Valor observado
SUCI	formato IMSI, 001/01, RI 0000, esquema nulo, MSIN 0123456780
RAND	`6a9e82c734214c669a02c4b6a6bfce01`
AUTN	SQN⊕AK `b6cfed53b033`, AMF `8000`, MAC `eb0e59f200567fe3`
RES*	`2fb00cbedf66b7b967083ca5b99525e0`
ABBA	`0000`
Algoritmos NAS	128-NEA2 (cifrado), 128-NIA2 (integridad)
Mensajes NAS cifrados	7 de 8 protegidos
IP del UE (PFCP)	10.45.0.5
TEID N3 del UPF	`0x0000cd8a` en 10.53.0.3 (NGAP y PFCP)
TEID N3 del gNB	`0x00000001` en 10.53.0.4 (NGAP y PFCP)
QoS	5QI 9, QFI 1
Reglas PFCP	PDR 1–4, FAR 1–3, QER 1, URR 1

Tabla VI
DURACIÓN DE LOS PROCEDIMIENTOS DE SEÑALIZACIÓN (MS, MEDIA ± IC 95 %, $n = 5$)

Procedimiento	**AS `nea2`**	**AS `nea0`**
NG Setup	18.7 ± 7.0	—
Autenticación 5G-AKA	29.9 ± 5.0	35.9 ± 12.8
Contexto inicial (N2)	76.2 ± 10.1	98.0 ± 30.6
PFCP Session Establishment	1.4 ± 0.8	—
Recursos de la sesión PDU (N2)	126.4 ± 35.4	120.5 ± 45.6
Registro hasta contexto	196.0 ± 18.7	306.3 ± 107
Registro hasta sesión PDU	$\mathbf{471.6 \pm 32.3}$	598 ± 133

control, por ejemplo para los mensajes de descubrimiento de enrutador.

La respuesta trae el F-TEID asignado por el UPF (`0x0000cd8a`, 10.53.0.3), que el AMF envía al gNB en la *PDUSessionResourceSetupRequest* junto con la QoS (5QI 9). El gNB contesta con su propio TEID (`0x00000001`, 10.53.0.4), y el SMF lo instala en el UPF con una *PFCP Session Modification*, que crea la cabecera GTP-U externa hacia el gNB [6]. El analizador comprueba que los dos TEID coinciden en NGAP y en PFCP. Una versión inicial tomaba por error la *Outer Header Creation* de la primera petición PFCP, que apunta al SMF (127.0.0.4, TEID 2) y no al gNB.

V-D. Tiempos de los procedimientos

La prueba de registro se repitió cinco veces con la seguridad AS en `nea2` y otras cinco con `nea0`. Cada corrida recrea la RAN, captura el registro y lo analiza. La Tabla VI presenta las medias con su intervalo de confianza del 95 % (t de Student, 4 grados de libertad).

Tres observaciones. Primera, los intervalos de `nea2` y `nea0` se solapan, así que cifrar RRC y PDCP no impone una penalización medible: la variabilidad la domina la carga del equipo. Segunda, el núcleo es rápido (la PFCP tarda unos 1.4 ms). Los procedimientos lentos son los que atraviesan la radio emulada: el contexto inicial y los recursos de la sesión PDU incluyen la reconfiguración RRC del UE sobre ZMQ. Esto coincide con la conclusión de la Fase 1 de que la latencia se origina en la ruta radio. Tercera, el total de unos 0.47 s es comparable con los 0.51 s medidos en la Fase 1 con una sola traza.

VI. BANCO DE PRUEBAS Y RESULTADOS

VI-A. Metodología

Las pruebas están implementadas una sola vez en un paquete de Python que solo usa la biblioteca estándar. Las acciones `manage_lab.ps1` (Windows) y `make` (Linux) solo lo invocan. Cada ejecución crea una carpeta de resultados con:

- `metadata.json`: equipo, CPU, *commit* de git, configuración sin las claves secretas, imágenes usadas y reinicios de contenedores;
- `resumen.csv`: una fila por métrica;
- las series detalladas y las salidas originales de ping, iperf3 y los logs, para auditar los cálculos.

Tabla VII
RESULTADOS DE LA CAMPAÑA (MEDIA ± IC 95 %, $n = 5$)

Prueba	**Métrica**	**Resultado**
P5	RTT medio al UPF (N6)	31.9 ± 0.9 ms
	RTT p95 al UPF	40.5 ± 1.5 ms
	RTT medio a Internet (8.8.8.8)	49.4 ± 4.5 ms
	Pérdida de paquetes	0 %
P6	TCP DL (MCS medio 27.7)	48.6 ± 1.4 Mbps
	TCP UL (MCS medio 6.4)	11.6 ± 0.5 Mbps
	BLER agregada DL / UL	0 % / 0.85 %
P7	UDP DL máximo	56.3 ± 1.2 Mbps
	UDP UL máximo	10.5 ± 0.2 Mbps
P10	CPU global (carga DL)	52.8 ± 2.0 %
	gNB / UE / núcleo	241 / 168 / 7.8 % de un núcleo
	Núcleos saturados (≥ 95 %)	0 de 8

Durante cada medición se registra también la tabla de métricas del gNB (CQI, MCS, SNR, ACK/NACK). Para cada métrica se calculan la media, la desviación estándar muestral y el intervalo de confianza del 95 % con t de Student ($n-1$ grados de libertad). La BLER se agrega sumando los ACK/NACK de todas las corridas y no promediando porcentajes, porque una corrida con pocos bloques pesaría igual que una con muchos. Los intérpretes de las salidas tienen 40 pruebas unitarias con salidas reales, y una integración continua en GitHub las ejecuta en cada *push*.

Todas las mediciones se hicieron en el mismo equipo. El canal ZMQ es síncrono: si la CPU no procesa 23.04 MS/s, el tiempo emulado transcurre más lento que el real y el caudal y el RTT empeoran en la misma proporción. Por eso las cifras de distintos equipos no son comparables. También se comprobó que la memoria del anfitrión influye: una serie de P7 medida con Windows sin memoria libre dio una pérdida de 6.0 ± 10.0 % a 40 Mbps. Repetida con la memoria del gNB reducida, dio 0.21 ± 0.03 %. Solo se informa esta última.

VI-B. Latencia, caudal y recursos (P5, P6, P10)

La Tabla VII resume la campaña principal: cinco corridas por prueba, srsUE 26.10 y NAS NEA2.

El RTT hacia Internet supera al del UPF en unos 17 ms, lo que corresponde al tramo N6 más la red externa. El enlace ascendente está limitado por el srsUE: la SNR del PUSCH es de apenas 0.7–3.5 dB y el MCS ronda 6. Se trata de una limitación conocida del emulador, ya observada en la Fase 1, y no del núcleo. En P10 el gNB y el UE consumen 2.4 y 1.7 núcleos y el núcleo 5G apenas 0.08, sin ningún núcleo saturado. La virtualización de las funciones del núcleo resulta, por tanto, barata frente al procesamiento de la capa física.

VI-C. Saturación con UDP (P7)

La Fig. 3 muestra el caudal recibido frente a la tasa ofrecida. En el sentido descendente, el canal entrega sin pérdidas hasta 40 Mbps (0.21 % de pérdida). A partir de 50 Mbps se satura,

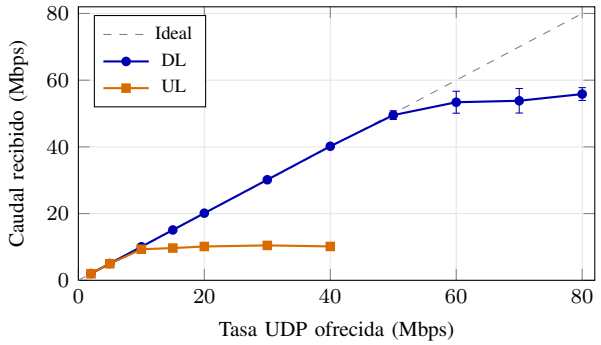


Figura 3. P7: caudal UDP recibido frente a la tasa ofrecida (media ± IC 95 %, $n = 5$). De 2 a 40 Mbps, campaña principal; de 40 a 80 Mbps en DL, barrido repetido con la red optimizada.

Tabla VIII
P8: CAUDAL TCP DL CON MCS FIJO FRENTE A LA COTA DE TS 38.306

MCS	Q_m	$R \cdot 1024$	Teórico	Medido ($n = 3$)	%
0	2	120	3.6	2.18 ± 0.07	61
5	2	379	11.3	6.75 ± 0.21	60
10	4	340	20.3	12.53 ± 0.07	62
15	4	616	36.9	21.01 ± 3.31	57
20	6	567	50.9	27.61 ± 3.29	54
25	6	822	73.8	40.81 ± 2.49	55
28	6	948	85.1	47.83 ± 3.31	56

con un máximo de 56.3 ± 1.2 Mbps, el 66 % de la cota teórica, y la pérdida crece hasta el 31 % a 80 Mbps. En el ascendente la saturación llega a unos 10 Mbps, con una pérdida inferior al 1 % solo hasta 5 Mbps.

VI-D. Caudal frente a MCS (P8)

En P8 se fija el MCS del PDSCH (mínimo igual a máximo) y se recrean el gNB y el UE para cada valor. El gNB aplicó exactamente el MCS configurado [8] en las tres corridas, con BLER descendente del 0 %. La cota teórica de cada MCS se calcula con TS 38.306 [9]:

$$R_{\text{máx}} = 10^{-6}\, \nu\, Q_m\, R\, \frac{12\, N_{PRB}}{T_s^{\mu}}\, (1 - OH), \qquad (1)$$

con una capa ($\nu = 1$), $N_{PRB} = 106$, $T_s^{\mu} = 10^{-3}/14$ s y $OH = 0.14$. La Fig. 4 y la Tabla VIII comparan ambos valores.

Análisis de la brecha. Tras la campaña se planteó como hipótesis que la diferencia con la teoría se debía al procesamiento en tiempo real (CPU). P8 la desmiente como causa principal: la fracción medida/teórica es casi constante, entre el 54 y el 62 %, en todos los MCS, incluso en MCS 0 con 2 Mbps, donde la CPU no puede limitar (P10 tampoco muestra núcleos saturados). Una fracción constante indica una sobrecarga estructural mayor que el 14 % que supone la norma para FR1. Entre sus componentes probables están el PDCCH, los símbolos DMRS, el SSB y los SIB, los slots que el planificador no asigna a datos y las cabeceras TCP/IP/GTP-U, además de la dinámica de TCP. La ligera caída del 60

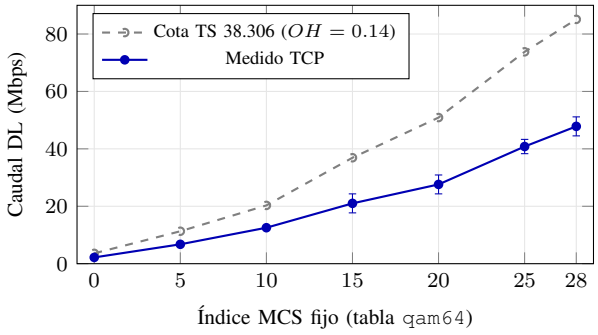


Figura 4. P8: caudal TCP descendente con MCS fijo frente a la cota teórica (media ± IC 95 %, $n = 3$).

al 55 % en los MCS altos sí podría deberse a la carga de procesamiento. Esto concuerda con lo que la Fase 1 ya planteó como hipótesis: la brecha no se explicaba por saturación global de la CPU. Cuantificar cada componente requiere el registro MAC del gNB (PRB y símbolos asignados por slot) y queda como trabajo futuro.

VI-E. Estabilidad (P9)

Durante 30 minutos se envió un ping por segundo al UPF con una carga UDP descendente de fondo de 5 Mbps. Se recibieron 1799 de 1800 respuestas (pérdida del 0.06 %), con RTT medio de 23.2 ms (p95 35.3 ms, p99 43.0 ms) y un único pico de 1.1 s. No hubo cortes, reinicios de contenedores ni recuperaciones automáticas, el gNB no registró *underflows*, *overflows* ni avisos de tiempo real, y la carga de 5 Mbps se entregó completa. El criterio de estabilidad se cumple: el gNB mantiene la conexión con el AMF y el túnel GTP-U sin microcortes.

VII. GESTIÓN TÉCNICA

VII-A. Flujo de trabajo

El proyecto se gestiona en un repositorio git con cuatro prácticas:

- una bitácora obligatoria (`BITACORA.md`) en la que cada cambio registra qué se hizo, por qué, cómo se verificó y qué no se pudo verificar, junto con una lista viva de pendientes;
- un README que describe en todo momento el estado y el modo de uso;
- pruebas unitarias;
- una integración continua en GitHub Actions que, en cada *push*, ejecuta las pruebas en Python 3.10 y 3.12, `shellcheck`, `yamllint` sobre las plantillas ya renderizadas y la validación de Docker Compose.

Las conclusiones técnicas se apoyan en evidencia citable (carpeta de resultados, línea de log o trama), y cuando una medición desmiente una conclusión previa, la corrección queda registrada, como ocurrió con P8. El repositorio se trasladó de una unidad en la nube a un disco local porque la sincronización impedía ver en Windows lo que escribían los contenedores (F2-14) y había dañado el directorio `.git`.

VII-B. Decisiones técnicas

La Tabla IX recoge las decisiones principales con su justificación.

VII-C. Fallas encontradas y su solución

Las quince fallas de la Tabla X se encontraron al validar la contenerización y las mediciones. Cada una tiene su diagnóstico detallado en la bitácora técnica del repositorio.

VII-D. Optimización de recursos

Una compilación de srsRAN se interrumpió por falta de memoria con la red levantada. Se identificaron tres causas: el *buffer pool* de 2 GiB del gNB, `make` con 8 trabajos simultáneos y la compilación en paralelo del gNB y del UE. Las medidas tomadas fueron:

- reducir el *buffer pool* a 256 MiB;
- calcular el número de trabajos de `make` según la memoria disponible, a unos 1.5 GiB por trabajo;
- compilar el gNB y el UE en serie, deteniéndolos antes;
- usar `ccache`;
- compilar el gNB una sola vez, con ZMQ y UHD;
- compilar del UE solo el binario y el controlador de radio que se usan.

Con estos cambios el gNB compiló en unos 16 min con dos trabajos, el UE completo en 193 s, y la red pasó de unos 3.6 a 1.85 GB.

VII-E. Preparación de la Fase 3

Sin hardware se dejaron listos y verificados:

- el servicio `gnb-sdr` con la misma imagen del gNB;
- dos plantillas para la USRP B210, en n3 FDD y en n78 TDD (20 MHz, 30 kHz, patrón de 10 slots con 6 DL + 8 símbolos y 3 UL), con sus parámetros comprobados contra el código del *commit* fijado. Con ellas el gNB acepta la configuración completa y solo falla al no encontrar la radio;
- el alta de la SIM física sin exponer K ni OPc;
- la calculadora de presupuesto de enlace, con la fracción de tiempo TDD;
- un plan de pruebas con criterios de aceptación, lista de verificación del hardware y riesgos.

El plan fija las pruebas conducidas, con atenuadores o caja de apantallamiento, y el PLMN de pruebas hasta contar con el permiso de MinTIC para emisiones al aire.

VIII. Conclusiones

1. El núcleo Open5GS quedó totalmente funcional y reproducible: once funciones de red, MongoDB aprovisionado con el perfil 5G del suscriptor (5G-AKA con Milenage, SST 1, 5QI 9) y seguridad NAS con cifrado 128-NEA2 e integridad 128-NIA2. Habilitar el cifrado requirió diagnosticar un defecto de descifrado del emulador y actualizarlo.
2. El gNB de srsRAN se vincula al núcleo por NGAP/SCTP (18.7 ± 7.0 ms de NG Setup) y mantiene estable el túnel GTP-U: treinta minutos de tráfico sin cortes ni reinicios, con TEID coherentes entre NGAP y PFCP.
3. El análisis de las trazas del registro y de la sesión PDU, automatizado con *tshark*, interpreta cada campo relevante (SUCI, RAND, AUTN, RES*, ABBA, algoritmos, TEID, reglas PDR/FAR/QER, IP) y mide cada procedimiento. El registro completo tarda 471.6 ± 32.3 ms, dominado por los pasos que cruzan la radio emulada. La PFCP es casi instantánea y cifrar RRC/PDCP no añade una demora medible.
4. Las mediciones con rigor estadístico dan un RTT de 31.9 ± 0.9 ms al UPF y caudales TCP de 48.6 ± 1.4 (DL) y 11.6 ± 0.5 Mbps (UL), con un máximo UDP DL de 56.3 ± 1.2 Mbps. El barrido de MCS muestra un caudal del 54–62 % de la cota teórica en todos los MCS, lo que atribuye la brecha a la sobrecarga de capa física y de planificación y no a la CPU. El UL lo limita el emulador srsUE.
5. La gestión técnica dejó trazabilidad completa: quince fallas resueltas con causa y corrección, decisiones justificadas, bitácora, pruebas unitarias e integración continua. La optimización de memoria redujo a la mitad el consumo de la red y estabilizó las mediciones.
6. **Trabajo futuro:** cuantificar la sobrecarga de capa física con el registro MAC del gNB, evidenciar el algoritmo AS negociado y, en la Fase 3, integrar la USRP B210 y el terminal comercial con la configuración y el plan ya preparados.

Referencias

[1] 3GPP, "System architecture for the 5G System (5GS); Stage 2," TS 23.501.
[2] 3GPP, "Procedures for the 5G System (5GS); Stage 2," TS 23.502.
[3] 3GPP, "Non-Access-Stratum (NAS) protocol for 5G System (5GS); Stage 3," TS 24.501.
[4] 3GPP, "NG-RAN; NG Application Protocol (NGAP)," TS 38.413.
[5] 3GPP, "Interface between the Control Plane and the User Plane nodes (PFCP)," TS 29.244.
[6] 3GPP, "General Packet Radio System (GPRS) Tunnelling Protocol User Plane (GTPv1-U)," TS 29.281.
[7] 3GPP, "Security architecture and procedures for 5G System," TS 33.501.
[8] 3GPP, "NR; Physical layer procedures for data," TS 38.214.
[9] 3GPP, "NR; User Equipment (UE) radio access capabilities," TS 38.306.
[10] 3GPP, "Specification of the MILENAGE algorithm set," TS 35.206.
[11] R. Stewart, "Stream Control Transmission Protocol," IETF RFC 4960, 2007.
[12] Software Radio Systems, "srsRAN Project documentation." [Online]. Available: https://docs.srsran.com
[13] Software Radio Systems, "srsRAN 4G (srsUE)," release 26.10. [Online]. Available: https://github.com/srsran/srsRAN_4G
[14] Open5GS, "Open5GS: Open source implementation of 5GC and EPC," v2.8.0. [Online]. Available: https://open5gs.org
[15] ZeroMQ, "ZeroMQ: distributed messaging." [Online]. Available: https://zeromq.org
[16] Docker Inc., "Docker Compose documentation." [Online]. Available: https://docs.docker.com/compose/
[17] Wireshark Foundation, "tshark: terminal-based Wireshark," v4.6. [Online]. Available: https://www.wireshark.org
[18] Congreso de Colombia, "Ley 1581 de 2012 (protección de datos personales)."

Tabla IX
REGISTRO DE DECISIONES TÉCNICAS DE LA FASE 2

Decisión	Justificación	Alternativa descartada
Contenedores separados con Docker Compose	Reproducible en cualquier equipo, con dependencias y orden de arranque declarados y reinicio automático	Instalación manual en WSL2 de la Fase 1 (no reproducible)
Open5GS v2.8.0 compilado desde fuente	Versión fija y sin el paquete `open5gs-webui`, que no existe en apt (F2-1)	PPA `latest`, que cambia de versión sin aviso
Configuración única en `.env` con plantillas	Un valor se cambia en un solo lugar; las claves de la SIM quedan fuera de git	Valores repetidos en cada archivo YAML
srsUE 26.10 para habilitar NEA2	Corrige el descifrado NAS de 5G (F2-15); se compiló sin sustituir la imagen anterior y se verificó en la traza	Mantener NEA0 (NAS sin cifrar)
Recrear gNB y UE en lugar de reiniciarlos	El reinicio en sitio dejaba el ZMQ sin muestras (F2-13, 5 de 5 veces)	`docker compose restart`
Banco de pruebas en Python sin dependencias	Una sola implementación, resultados trazables y estadística con IC 95 %	Scripts separados en PowerShell y bash
Buffer pool del gNB de 256 MiB	72 % menos memoria sin pérdida de caudal	Valor por defecto de 2 GiB
Una imagen del gNB con ZMQ y UHD	Una sola compilación de srsRAN para la Fase 2 y la Fase 3	Imagen separada para SDR (segunda compilación)
Compilación acotada en memoria	Trabajos de `make` según la memoria disponible, `ccache` y compilación en serie: la compilación ya no se queda sin memoria	`make -j$(nproc)` con la red levantada

Tabla X
FALLAS DE LA FASE 2: SÍNTOMA, CAUSA Y CORRECCIÓN

#	Síntoma	Causa	Corrección
F2-1	La imagen del núcleo no construía	`open5gs-webui` no existe como paquete apt	WebUI en su contenedor, compilada desde fuente
F2-2	UDR/PCF sin base de datos	YAML del paquete con `mongodb://localhost`	El *entrypoint* apunta al contenedor `mongodb`
F2-3	PLMN 999/70 o configuración rechazada	Formatos antiguos y nuevos mezclados en `amf.yaml`/`nrf.yaml`	YAML reescritos en formato 2.8 con PLMN 001/01
F2-4	GTP-U inalcanzable	UPF en 0.0.0.0 y gNB en 127.0.0.1	Ambos enlazan su IP de la red Docker
F2-5	El gNB esperaba al AMF indefinidamente	Sondeo de NGAP por TCP (NGAP usa SCTP)	*Healthcheck* sobre SCTP y `service_healthy`
F2-6	Sin radio entre gNB y UE	Puertos ZMQ cruzados	Cada extremo enlaza su TX y se conecta al TX del otro
F2-7	Ping a Internet “funcionaba” sin 5G	La ruta por defecto de Docker ganaba	Ruta por defecto por `tun_srsue`, vigilada
F2-8	Sesión PDU rechazada (“No 5QI”)	Suscriptor con el esquema antiguo (`pcn`)	Esquema de Open5GS 2.8 (`qos`)
F2-9	Caudal bajo y cola de logs desbordada	PHY/MAC en `info` y PCAP de N3 en el gNB	Logs en `warning`; N3 capturado en el núcleo
F2-10	Scripts rotos al clonar en Windows	Conversión a CRLF de los `.sh`	`.gitattributes` y normalización en los Dockerfiles
F2-11	PCAP corrupto desde PowerShell	La redirección > reescribe el binario en UTF-16	`tcpdump -w` dentro del contenedor
F2-12	*Pool overlaps* al levantar la red	Subred 172.20.0.0/24 en el rango de otros proyectos	Subred 10.53.0.0/24 y verificación previa automática
F2-13	UE en *Attaching* tras reiniciar la RAN	`compose restart` deja el ZMQ sin muestras	Recrear gNB y UE juntos
F2-14	Capturas invisibles en Windows	Repositorio en una unidad de Google Drive	Copia con `docker compose cp` y repositorio en disco local
F2-15	UE liberado tras el *Security Mode Command* con NEA2	srsUE 25.10 descifraba el NAS de 5G con la NAS COUNT de LTE	srsUE 26.10 (`06f0b1e23`)